

Conv-Tasnet

Encoder

Mục tiêu:

- Chuyển đổi các đoạn chồng lẫn nhau của đầu vào thành không gian đặc trưng cao chiều hơn.
- Đầu ra của bộ mã hóa được tối ưu hóa để tách các nguồn âm thanh riêng lẻ thông qua các bộ lọc đã học.

Biểu diễn toán học:

$$\mathbf{w} = \mathcal{H}(\mathbf{x}\mathbf{U})$$

- $\mathbf{U}$: Ma trận chứa N hàm cơ sở (basis functions) của bộ mã hóa, mỗi hàm có độ dài L .
- $\mathcal{H}$: Hàm phi tuyến đảm bảo đầu ra không âm. (trong code mẫu là ReLU)
- $\mathbf{x}$: Âm thanh được chia thành các đoạn chồng lấp.

Các bước thực hiện:

- **Phân đoạn**: Sóng âm được chia thành các đoạn chồng lấp có độ dài L , với bước $L/2$ (chồng lấp 50%).

```
class Encoder(nn.Module):
    def __init__(self, L, N):
        super(Encoder, self).__init__()
        self.L, self.N = L, N # L: độ dài bộ lọc, N: số bộ lọc
        self.conv1d_U = nn.Conv1d(1, N, kernel_size=L, stride=L
```

- **Tích chập 1 chiều**: Áp dụng N bộ lọc kích thước L lên từng đoạn để tạo biểu diễn đặc trưng nhiều chiều.
- **Hàm phi tuyến (ReLU)**: Đảm bảo các giá trị đầu ra không âm.

```
def forward(self, mixture):
    mixture = torch.unsqueeze(mixture, 1) # Thêm chiều kênh: [M, 1, N, K]
    mixture_w = F.relu(self.conv1d_U(mixture)) # Tích chập và F.relu
    return mixture_w # [M, N, K]
```

Đầu ra:

- Đầu ra của bộ mã hóa là mixture có kích thước [M,N,K]
 - M: Kích thước batch.
 - N: Số bộ lọc (đặc trưng).
 - $K=2T/L-1$: Số khung thời gian từ phép tích chập.
 - T là độ dài của biểu diễn đặc trưng K

Decoder

Mục tiêu: Áp dụng mặt nạ lên biểu diễn đặc trưng và tái tạo các nguồn âm thanh đã tách bằng cách sử dụng các hàm cơ sở học được.

Các bước thực hiện:

- **Áp dụng mặt nạ:**
 - Các mặt nạ mi được áp dụng lên biểu diễn w để tách các đặc trưng của từng nguồn

```
source_w = torch.unsqueeze(mixture_w, 1) * est_mask
```

- **Giải mã đặc trưng:**
 - Các đặc trưng được chuyển về miền thời gian bằng các hàm cơ sở học được.

```
est_source = self.basis_signals(source_w)
```

- **Tổng hợp đoạn chồng lấp:**

- Các đoạn chồng lấp được tổng hợp để tạo thành sóng âm liên tục cho từng nguồn.

```
est_source = overlap_and_add(est_source, self.L // 2)
```

Đầu ra:

- Bộ giải mã trả về sóng âm tái tạo với kích thước $[M,C,T]$, trong đó:
 - M: Kích thước batch.
 - C: Số lượng nguồn âm thanh.
 - T: Độ dài sóng âm tái tạo.

Estimating the separation masks

Mục đích: ước lượng các mặt nạ để tách các nguồn âm thanh từ tín hiệu hỗn hợp. Các mặt nạ được áp dụng trực tiếp lên biểu diễn đặc trưng của bộ mã hóa để cô lập từng nguồn riêng lẻ.

Các bước thực hiện:

1. Định nghĩa mặt nạ:

- Các mặt nạ m_i là các vector trong $R^{(1 \times N)}$, với N là số đặc trưng của bộ mã hóa.
- Mỗi mặt nạ tương ứng với một nguồn/người nói trong tín hiệu hỗn hợp.
- Mặt nạ được áp dụng từng phần tử lên đầu ra của bộ mã hóa w để cô lập đặc trưng của từng nguồn.

$$\mathbf{d}_i = \mathbf{w} \odot \mathbf{m}_i$$

- w : Biểu diễn đặc trưng từ bộ mã hóa.
- m_i : Mặt nạ cho nguồn thứ i .
- d_i : Đặc trưng của nguồn thứ i .

```
M, N, K = mixture_w.size() # Lấy kích thước đầu vào
score = self.network(mixture_w) # Truyền qua mạng TCN
score = score.view(M, C, N, K) # Reshape to match mask dimension
est_mask = F.relu(score) # Apply non-linearity to constrain mask
```

2. Tái tạo nguồn:

- Sau khi áp dụng mặt nạ lên đầu ra của bộ mã hóa, bộ giải mã sẽ tái tạo sóng âm cho từng nguồn.

$$\hat{s}_i = \mathbf{d}_i \mathbf{V}$$

- $\mathbf{d}_i$: Biểu diễn đặc trưng của nguồn i .
- $\mathbf{V}$: Hàm cơ sở học được trong bộ giải mã.
- $\hat{s}_i$: Sóng âm tái tạo của nguồn i .

```
source_w = torch.unsqueeze(mixture_w, 1) * est_mask
```

Convolutional separation module

Module tách được xây dựng dựa trên **Mạng Tích chập Thời gian (Temporal Convolutional Network - TCN)**. Các đặc điểm chính của kiến trúc này bao gồm:

1. Tích chập giãn cách (Dilated Convolution):

- Các lớp tích chập sử dụng yếu tố giãn cách tăng dần theo cấp số nhân ($d=2^l$), cho phép mở rộng trường tiếp nhận mà không tăng chi phí tính toán.
- Điều này giúp mạng mô hình hóa cả phụ thuộc ngắn hạn và dài hạn trong tín hiệu âm thanh.

2. Lặp lại (Repeats):

- Một chuỗi các khối tích chập được sắp xếp thành R lần lặp. Mỗi lần lặp bao gồm X khối tích chập.

3. Kết nối dư và bỏ qua (Residual and Skip Connections):

- Kết nối dư giữa các khối giúp ổn định quá trình huấn luyện và cải thiện việc lan truyền gradient.
- Kết nối bỏ qua giúp tổng hợp thông tin giữa các khối và tăng hiệu suất học.

```
# Xây dựng các khối tích chập thời gian
repeats = []
for r in range(R): # Lặp lại các khối (R lần)
    blocks = []
    for x in range(X): # X khối trong mỗi lần lặp
        dilation = 2**x # Tăng giãn cách theo cấp số nhân
        padding = (P - 1) * dilation // 2 # Bù đắp để giữ nguyên
        blocks += [TemporalBlock(B, H, P, stride=1, padding=padding)]
    repeats += [nn.Sequential(*blocks)]

temporal_conv_net = nn.Sequential(*repeats) # Kết hợp các khối
```